

3.1 From Recurrence to Attention

The seq2seq bottleneck, then a path of length one at cost T squared.

These notes match the lecture slides. Use **(slides)** on the course hub for the deck.

An RNN compresses the whole past into one vector. Token 1 can affect token 40 only by surviving 39 overwrites. **Attention** lets every token look at every other token in one step. That is the move from Week 2's recurrences to the rest of the course.

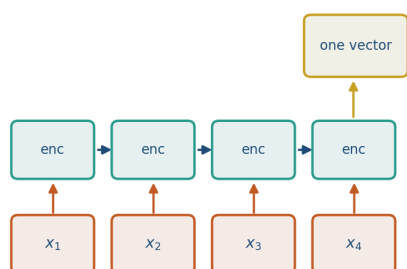
First time this method appears. **Attention** is a path of length one between any two positions.

What. Instead of squeezing a sentence into one RNN hidden state, each output position **looks at** all encoder states with a weighted sum. **Why.** Seq2seq RNNs have a bottleneck: one vector for the whole source. Long sources forget the beginning. **Architecture.** Encoder states $h_1, \dots, h_T$. Decoder query q_t . Weights over source, then mix. Self-attention (next note) uses the same sequence for query and keys. **How.** Score, softmax, weighted sum. Cost is T^2 scores, not path length T . **Formula.** Path length 1 at cost $O(T^2 d)$. Bahdanau: $e_{ti} = v^T \tanh(W_q q_t + W_h h_i)$. **Tradeoffs.** + No sequential bottleneck, interpretable weights. – Quadratic cost in T ; you still need the formula next hour.

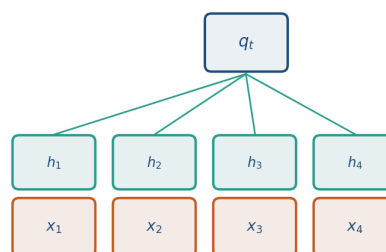
1. The bottleneck

A seq2seq RNN encodes the source into **one** vector, then the decoder must generate from that vector. Long source, same-size bottle. CS224N's slogan: attention is a **direct connection** from the decoder to the encoder states, so the model can reread instead of remember.

Seq2seq bottleneck: the source is one vector



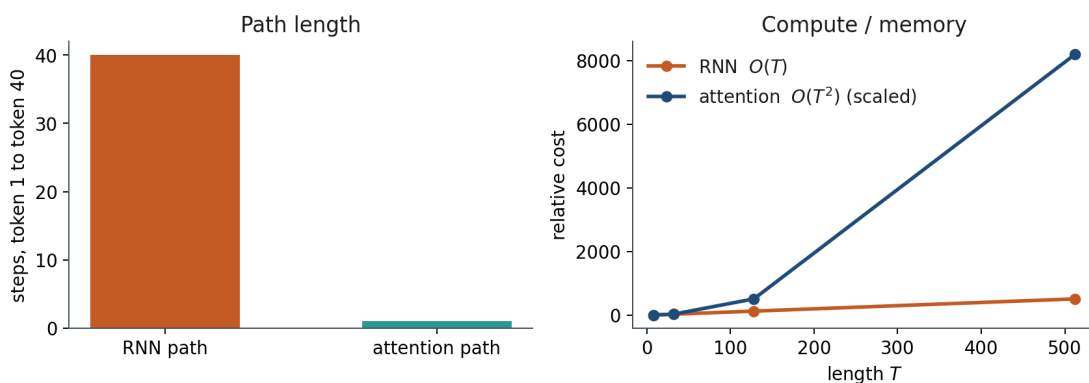
Attention: the decoder rereads the source



Bahdanau et al. (2015) added that reread **on top of** an RNN. Queries come from the decoder; keys and values come from the encoder. Vaswani et al. (2017) dropped the recurrence: **self-attention** is the layer, and every token in the **same** sequence plays all three roles.

Draw *The cat ... it* with 40 dots. RNN: 39 arrows in a line. Attention: one arrow from *it* to *cat*, plus arrows to everyone else, with **weights**. Note 3.2 turns those weights into softmax.

2. Path length 1 is not cost 1



	RNN	Self-attention
Path length, token 1 to T	T	1
Parallel over time	no	yes
Cost in T	$O(T)$	$O(T^2)$ memory/time for full attention

The path-length win is why Week 2's Jacobian product stops being the story. The T^2 bill is why Week 7 exists. For this course's default, **full attention** is the model.

Parallel over time: given all embeddings X , you form Q, K, V with matrix multiplies a GPU likes. An LSTM must finish step $t - 1$ before step t . That, more than "transformers are magic," is why pretraining scaled.

Write T and T^2 for $T = 32, 512, 4096$. Sparse or sliding-window attention is what you do when T^2 no longer fits. You still teach full attention first.

FlashAttention changes the **implementation**, not the fact that every pair can interact. Decoder-only GPT still pays $O(T^2)$ in training. At decode time, caching keys and values makes each new token $O(T)$ against the past. That is inference, not the training flop count.

Keep the slogan: **RNN path T , attention path 1, attention cost T^2 .**

3. Teaching this note

~16 minutes. Bottleneck picture, Bahdanau vs self-attention in one sentence, then the path-length table and the $T = 4$ vs $T = 4096$ arithmetic. Play **0:00–12:00** of 3Blue1Brown's GPT visual intro (next-token prediction and the first all-to-all cartoon). Do **not** derive QKV here; that is note 3.2.

4. Worked example

Sentence length $T = 4$ tokens: `cat`, `sat`, `on`, `it`.

RNN path from `cat` to `it`: 3 steps. Attention path: **1** score in the last row of a 4×4 matrix.

Full attention stores $T^2 = 16$ scores. For a 4k-token PDF, $T = 4096$, $T^2 \approx 1.68 \times 10^7$ scores **per head per layer**. If one score is 4 bytes, that is about 67 MB for **one** map, before batching and before stacking layers. That is the quadratic complaint.

GPU parallel: four tokens' queries can be dotted with keys in one matmul. LSTM: four sequential cell calls.

If each RNN Jacobian factor is 0.9, the product over 39 steps is $0.9^{39} \approx 0.017$. Attention's hop is 1. That is Week 2 meeting Week 3.

5. Where students get stuck

- Hearing "path length 1" and thinking attention is $O(1)$ compute (it is $O(T^2)$).
 - Mixing Bahdanau (attention **on** an RNN) with Vaswani (attention **instead of** an RNN).
 - Believing the RNN state "contains the whole sentence" in a lossless way.
-

6. Video

Watch [3Blue1Brown: But what is a GPT? Visual intro to transformers](#).

Pause when next-token prediction is a stack of vectors, and at the first all-to-all attention cartoon. Stop before the full QKV derivation if you are teaching 3.2 next.

The matching university lecture for *form* is Stanford CS224N W26 Lecture 5 (attention and transformers): bottleneck, then Bahdanau, then self-attention. We do not copy those slides. The transformer block waits for 3.3.

7. Practice

1. Why can a transformer GPU-batch a whole sentence's updates at once, while an LSTM cannot?
2. A 4k-token policy PDF: what is the quadratic cost complaining about?
3. In one sentence, what information did h_t have to store that an attention row can fetch instead?
4. For $T = 8$, how many attention scores are in one head's $T \times T$ map? For $T = 32$, how many? What is the ratio $32^2 / 8^2$?
5. An RNN of length $T = 50$ has path length 49 from token 1 to token 50. Write the attention path length. If each RNN Jacobian factor is 0.9, bound the RNN product 0.9^{49} vs attention's single hop of 1.
6. Bahdanau versus Vaswani, in one sentence each: where do the queries come from?
7. Caching keys and values at decode time makes each new token cheaper. Does that change the **training** cost of a full attention map? Why or why not?